REPORT · 2026年8月

造像类文物三维建模自动化系统 UV纹理+并发+HTML报表 开发说明书

UV 图集纹理（xatlas）· 并发调度（进程池）· HTML 报表（jinja2）

文保数字化研发组
2026年8月

目 录

一、概述 3

二、模块划分 4

三、依赖关系与数据流 5

3.1 依赖方向 5

3.2 UV 纹理数据流 5

3.3 并发调度数据流 5

四、模块详细设计 7

4.1 texture/uv.py — UV 图集纹理 7

4.2 并发调度 — ProcessPoolExecutor 7

4.3 report/html_export.py — HTML 报表 7

五、健壮性与异常处理 8

六、测试与验证 9

七、开发进度与下一步方向（更新） 10

7.1 完成度（三项增强已交付） 10

7.2 下一步方向（更新） 10

附录 A：UV 图集产物示例 11

一、概述

本文档是《造像类文物三维建模自动化系统（路线 E）》**三项增强开发**的代码说明书：**① UV 图集纹理**（M4 texture.paint_mode='uv' 实装，xatlas 展开 + 图集烘焙）、**② 并发调度**（runner.workers 进程池并行）、**③ HTML 报表**（report.formats 增加 html，jinja2 + base64 内嵌图表）。三者对应测试指南 §6.1 的 P1/P2 项，为系统从「实验室可用」走向「业务试用」补齐关键能力。

> 三块能力一句话：**UV 纹理**让带色模型从「顶点色」升级为「真实纹理图集」（OBJ+MTL+PNG / GLB 内嵌），M5 的 overlap 指标随之具备启用条件；**并发**让多尊批量处理按造像并行（workers=2 实测 3 尊全链路完成），主进程统一写 manifest 杜绝写冲突；**HTML 报表**输出单文件自包含报表（指标 + 状态 + 失败清单 + 内嵌图表），可离线分享、随交付包归档。

二、模块划分

```
src/h3d/texture/uv.py        # 新增：UV 展开 + 图集烘焙 + 带纹理导出
   +-- unwrap_mesh     # xatlas.parametrize -> (新网格, UV, 面索引)
   +-- bake_texture    # 点云颜色 -> UV 图集（重心采样 + 光栅化）
   +-- attach_texture  # triangle_uvs + textures 附着
   +-- export_textured # OBJ(+MTL+PNG) / GLB 带纹理导出

src/h3d/report/html_export.py  # 新增：HTML 报表（jinja2 内嵌模板）
   +-- generate_html   # 单文件自包含报表（图表 base64 内嵌）

src/h3d/runner/batch_processor.py  # 增强：并发调度
   +-- _run_task_worker  # 子进程 worker（不写共享 manifest）
   +-- BatchProcessor._run_parallel  # ProcessPoolExecutor 并行
```

模块文件	核心职责	对外关键符号
texture/uv.py	UV 图集纹理（xatlas 展开 + 烘焙 + 导出）	unwrap_mesh, bake_texture, attach_texture, export_textured
texture/__init__.py	paint_mode='uv' 接入 run()（失败降级顶点色）	run（uv_baked 指标）
report/html_export.py	HTML 报表（jinja2 内嵌模板 + base64 图表）	generate_html
report/__init__.py	formats 支持 html（与 csv/png 组合）	generate
runner/batch_processor.py	并发调度（workers>1 进程池，主进程写 manifest）	_run_task_worker, BatchProcessor.run
core/config.py	ReportConfig 增 project_name；formats 默认含 html	ReportConfig

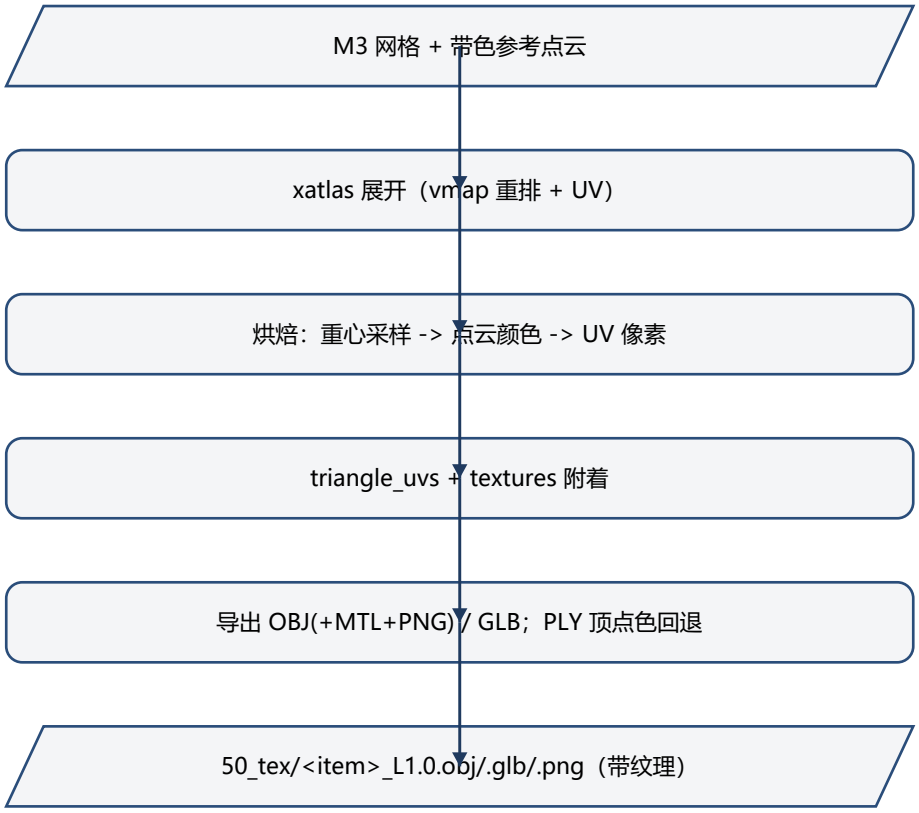
三、依赖关系与数据流

3.1 依赖方向

三项增强的依赖：UV 引入 xatlas（MIT，UV 展开）；HTML 引入 jinja2（BSD，模板引擎）；并发仅用标准库 concurrent.futures（ProcessPoolExecutor）。全部为白名单许可，不引入GPL 依赖：

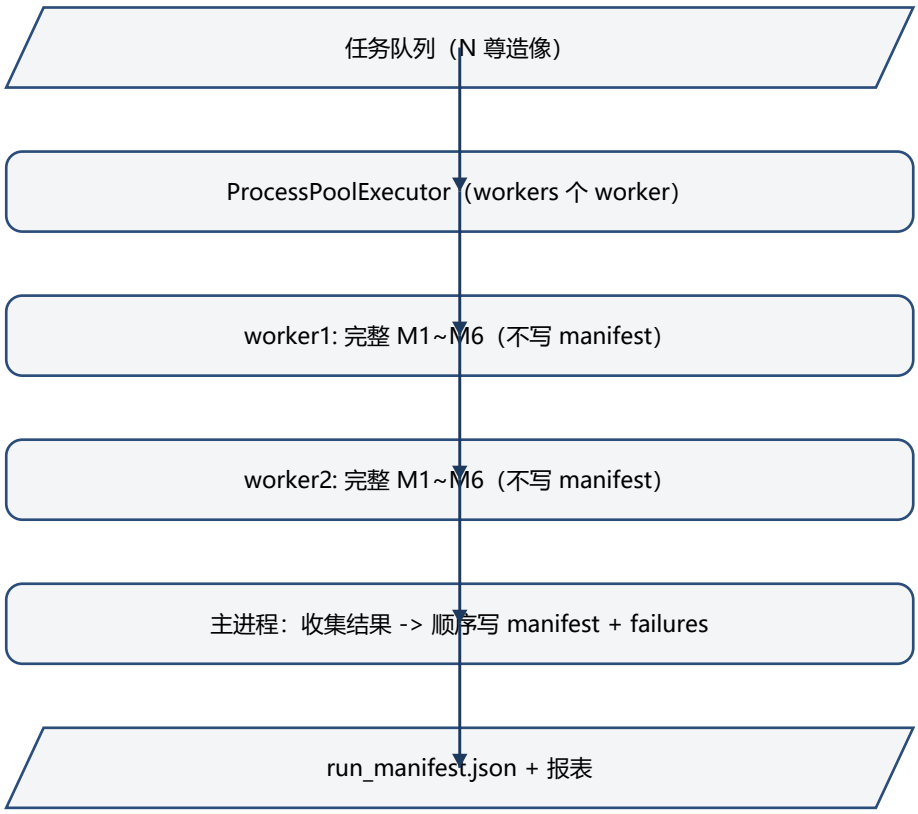
```
package-level deps:
  texture.uv --> xatlas(MIT) + scipy(B 重邻) + Open3D
  report.html_export --> jinja2(BSD) + base64/stdlib
  runner.batch_processor --> concurrent.futures(stdlib)
  texture / report / runner 均不反向依赖其它业务子包
```

3.2 UV 纹理数据流



UV 纹理数据流：xatlas 展开 -> 点云颜色烘焙到 UV 图集 -> 带纹理导出

3.3 并发调度数据流



并发调度：worker 只算不写（避免多进程并发写同一 manifest），主进程统一落盘

四、模块详细设计

4.1 texture/uv.py — UV 图集纹理

设计书 §35 的 uv_method=xatlas 实装：``xatlas.parametrize`` 返回 (vmap, indices, uvs) 三元组，按 vmap 重排顶点、indices 重建三角形、uvs 为每顶点 UV（[0,1]）。纹理烘焙把参考点云颜色映射到 UV 图集：每三角形在 3D 侧重心采样 8 点，scipy.cKDTree 取最近邻点云颜色；UV 侧光栅化到纹理像素。导出时 Open3D 设置 triangle_uvs + textures，OBJ 自动写 MTL + PNG，GLB 内嵌纹理；PLY 不支持纹理时回退顶点色（保持交付包完整）。

```
def unwrap_mesh(mesh):
    vmap, indices, uvs = xatlas.parametrize(verts, faces)
    new_mesh = TriangleMesh(verts[vmap], indices)  # 保留顶点色/法线
    return new_mesh, uvs, indices

def bake_texture(mesh, uv, ref_pcd, size=1024):
    # 每三角形：重心采样 8 点 -> 3D 最近邻颜色 + UV 光栅化
    for tri in faces:
        pts3d = barycentric(verts[tri])       # (8, 3)
        colors = cKDTree(ref).query(pts3d)[1]  # 最近邻颜色
        image[py, px, :] = colors             # 光栅化填充
    return image  # (size, size, 3) uint8
```

4.2 并发调度 — ProcessPoolExecutor

BatchProcessor.run 在 ``workers>1 且任务数>1`` 时走并行路径：每个 worker 独立处理一尊的完整 M1~M6（``_run_task_worker``，模块级函数可 pickle），**不写共享 manifest**，返回序列化结果；主进程收集后用 ``_strip_reserved`` 过滤保留键、顺序写 manifest 与失败清单，杜绝多进程并发写同一文件的冲突（软著方案 2.8 的工程约束）。Windows spawn 兼容：worker与 stage 均要求模块级函数。worker 崩溃（BrokenProcessPool）由主进程兜底记录失败。

配置项	默认值	用途
runner.workers	1	>1 时按造像并行（进程池）；1 = 串行（测试/调试）
runner.fail_isolation	true	worker 内异常隔离语义（false 时异常向主进程传播）

4.3 report/html_export.py — HTML 报表

jinja2 内嵌字符串模板（无外部 .html 依赖，单文件可移植），输出自包含 HTML：项目/运行概要+ 各尊阶段状态表（M1~M6）+ M5 质检指标表（覆盖率/中误差/判定）+ M6 交付索引 + 失败清单+ 阶段耗时图（PNG base64 内嵌，离线可分享）。``report.formats`` 支持 [csv, png, html] 任意组合；ReportConfig 新增 ``project_name`` 供报表展示（CLI 自动传入 cfg.project.name）。

配置项	默认值	用途
report.formats	[csv, png, html]	报表格式（html 新增）
report.project_name	''	报表展示的项目名（CLI 自动填充）

五、健壮性与异常处理

场景	处理	说明
xatlas 展开失败（退化网格）	InputDataError -> texture.run 降级顶点色路径	软著方案 2.6「单项缺失 -> 注明」
无颜色点云 + paint_mode='uv'	降级顶点色路径（uv_baked=False）	不抛异常
PLY 导出带纹理	跳过 + 警告，顶点色回退	PLY 格式不支持纹理
metrics 含保留键（stage 等）	_strip_reserved 过滤	防 manifest.record 关键字冲突
闭包 stage（不可 pickle）	并行路径仅支持模块级函数；测试用模块级 mock	Windows spawn 约束
worker 崩溃	主进程兜底记录失败（failures + manifest）	BrokenProcessPool
多进程写 manifest	worker 不写，主进程统一写	杜绝写冲突

六、测试与验证

三项增强配套 18 项单元测试，全量 **252 项** 通过（234 + 18）：29 core + 20 io + 16 preprocess + 11 register + 21 mesh + 26 texture(含 6 UV) + 22 quality + 26 report(含 4 HTML) + 36 runner(含 5 并发) + 6 cli + 10 change + 14 colour。

测试文件	覆盖点
tests/test_texture/test_uv.py	UV 展开（UV 范围/顶点色保留/空网格 E100）、烘焙（尺寸/无颜色异常）、导出（obj/glb 带纹理、ply 跳过）、run uv 模式（烘焙/降级）
tests/test_runner/test_parallel.py	并行全任务/写 manifest/失败隔离（reason 字段）/单尊回退串行/workers=1 串行
tests/test_report/test_html.py	HTML 章节/内嵌图表 base64/generate 多格式组合/仅 html

```
$ python -m pytest -q
..................................................................... [ 33%]
..................................................................... [ 66%]
..................................................................... [100%]
252 passed, 8 warnings in 46.24s

$ python -m h3d.cli batch -c demo.yaml   # workers=2 + paint_mode=uv + formats=[csv,png,html]
批量处理完成：total=3 done=3 failed=0 skipped=0（workers=2）
报表产物 -> report/metrics_summary.csv
报表产物 -> report/stage_timings.png
报表产物 -> report/report.html

50_tex/rock001/ 产物（UV 图集）：
  rock001_L1.0.obj + rock001_L1.0.mtl + rock001_L1.0_0.png + rock001_L1.0.glb + rock001_L1.0.ply
```

七、开发进度与下一步方向（更新）

7.1 完成度（三项增强已交付）

能力	状态	说明
主线 M1~M6 + X1/X2	已交付	测试指南 §6.1 P1/P2 项之前的全部能力
UV 图集纹理（paint_mode='uv'）	**本次交付**	xatlas 展开 + 烘焙 + OBJ/GLB 带纹理
并发调度（workers）	**本次交付**	ProcessPoolExecutor 按造像并行，主进程写 manifest
HTML 报表	**本次交付**	jinja2 单文件自包含 + base64 内嵌图表
CloudCompare 交叉校验 / GUI / 打包发布	未开始	测试指南 §6.2~§6.6 方向

全量 252 项测试 100% 通过；许可证全部 MIT/BSD（新增 xatlas MIT / jinja2 BSD）；三块能力在 CLI 全链路（workers=2 + uv + html）端到端验证通过。

7.2 下一步方向（更新）

优先级	方向	目标	关键点
P1	M5 overlap 指标启用	UV 路径下计算纹理重叠度（设计书 §17.3 ≥30%）	UV 岛面积统计
P1	CloudCompare 交叉校验	change.validation 完整调用 CC CLI M3C2	外部可执行文件依赖
P2	X1/X2 接入编排	change/colour 条件注册为可选 stage	X1 需约定 t1/t2 目录规范
P2	GUI 原型（PySide6）	新建项目向导 + 批量处理仪表盘	测试指南 §6.3~§6.5 架构
P3	软件化打包发布	PyInstaller + 图标品牌化	测试指南 §6.6

建议路径：三项增强后系统已具备「业务试用」核心能力（真实纹理 + 并行吞吐 + 可分享报表）。下一步优先启用 M5 overlap 指标（UV 纹理的自然延伸），随后进入 GUI 原型开发（测试指南 §6.3~6.5 五区架构），最终打包发布（§6.6）。

附录 A：UV 图集产物示例

```
50_tex/rock001/
+-- rock001_L1.0.obj      # OBJ 网格（引用纹理）
+-- rock001_L1.0.mtl      # MTL 材质（map_Kd 指向 PNG）
+-- rock001_L1.0_0.png    # UV 纹理图集（1024x1024，点云颜色烘焙）
+-- rock001_L1.0.glb      # GLB（内嵌纹理，Web 直接加载）
+-- rock001_L1.0.ply      # 顶点色回退（PLY 不支持纹理）

run_manifest.json 中 M4 阶段条目（uv 模式）：
  {status: done, colored: true, uv_baked: true, paint_mode: uv,
   formats: [ply, obj, glb], lod_levels: [1.0], n_outputs: 5}
```